PA-consistency Turing machines: comparison, constructions, and reduction experiments

Technical report and reproducible research artifacts

13 November 2026

Abstract

The requested strict state-count target has not been achieved: this archive contains no accepted blank-start PA-consistency machine with fewer than 196 working states. It provides a pinned comparison of the supplied 208-, 209-, and 196-state families; two newly verified 196-state implementations of the diagonal-vacuity approach; verified scanner-oriented 196-state layouts; and reproducible certificates for the attempted further reductions. The diagonal implementation improves the incoming 199-state realization to 196 states. Separating its diagonal clause as a head shortens the AX word further, from 191 to 190 macro items, while retaining 196 physical states.

The comparison distinguishes thirty-seven supplied accepted presentations in thirty transition-graph classes. The hybrid of U208's outer program and PA196's AX passes every primitive source-frame contract at 196 states but fails blank startup. The report explains that obstruction, rejected optimistic 195-state counts, restricted allocation obstructions, and the exact scope of the bounded physical searches. It also separates the published PA-internal nonhalting-implies-consistency theorem from the additional written proof translation required for the full PA-provable biconditional. No new Lean-kernel theorem or general 196-state lower bound is claimed.

Reading the result. The exact positive tables are in the `axioms/diagonal196/`, `axioms/separate196/`, and `axioms/search/` directories of the archive. Each new diagonal-calculus layout has 392 transition rows, initial state 0, and a separate halt label 196. The top-level reproduction instructions distinguish these accepted machines from the rejected or incomplete experiments. A negative experimental result is included to make the boundary of the work reviewable, not as evidence that the user's target is impossible.

Contents

1 What is being compared **3**

1.1 Presentations, transition graphs, and source programs . . . 3

1.2 What changed between the branches . . . 4

1.3 The published formal theorem boundary . . . 5

2 New constructions that pass the complete checks **5**

2.1 Diagonal vacuity in 196 states . . . 5

2.2 Separating the diagonal head: 190 macro items . . . 6

2.3 Scanner-oriented 196-state layouts . . . 6

2.4 What the executable certificates establish . . . 7

3 The precise PA equivalence **7**

3.1 Coding and arithmetic interpretation . . . 8

3.2 Proof templates for finite separation . . . 8

3.3 The proof-store invariant and the zero alias . . . 9

3.4 Reflecting physical halting back to the source . . . 10

3.5 Why diagonal vacuity preserves PA strength . . . 10

3.6 The simpler Shift used by U208 . . . 11

3.7 What ConciseIPRA contributes . . . 11

4 Why the attempted additional state savings were rejected **12**

4.1 An optimistic 195-state count was not a construction . . . 12

4.2 Restricted allocation has explicit Hall obstructions . . . 12

4.3 The U208/PA196 hybrid and its startup obstruction . . . 13

4.4 Bounded physical state-fold searches . . . 13

4.5 Head-offset state identifications . . . 14

4.6 Finite register invariants . . . 14

5 Archive contents and reproduction **15**

5.1 Accepted constructions . . . 15

5.2 Experimental searches and allocation certificates . . . 16

5.3 Rebuilding the PDF . . . 16

5.4 Evidence levels and limitations . . . 16

A Literal diagonal196 transition table **17**

B Complete supplied-design catalogue **19**

1 What is being compared

All state counts in this report count working states. The distinct halt label is not counted. Machines have one two-way infinite tape, alphabet $\{0, 1\}$, an all-zero initial tape, a distinguished initial state, and one-cell left or right moves. The supplied dense tables have exactly two transition rows for every working state. A larger alphabet, nonblank advice tape, or an external input program is not allowed to account for a claimed reduction.

The comparison is pinned to the source snapshots in Table 1[1, 2, 3]. Dates alone are inadequate: the incoming directory includes cumulative archives containing earlier attempts, and the main BusyBeaver branch already contains the newer construction.

Source	Pinned revision
BusyBeaver, PA196	`407cdc356f6b586635e71ac92be394255f4bfea4`
BusyBeaver, PA209	`69d98241a721ead20b4f0b17df9062950b40fbfe`
ProveIt incoming and ConciseIPRA	`dc523e7b06ffb414093c90aa035500ca283e5f75`

Table 1: Repository snapshots actually inspected.

The PA196 dense transition file has SHA-256

`e8bbccd4cfa325251ca95066db116b5aa766e404b0971c33c02772be1c3617a8`.

The corresponding PA209 file has SHA-256

`0610d978675320f0b4b1ef34619dcd3daa5912632d7037e200dfcc70954fea51`.

The comparison receipts bind the inspected files to those revisions and bind extracted incoming files to their exact ZIP member bytes. This avoids treating similarly named files in overlapping archives as independent constructions.

1.1 Presentations, transition graphs, and source programs

A catalogued presentation here is an origin-identified table occurrence with a particular naming of its states. Byte-identical copies in different supplied archives remain separate catalogue entries, but are not counted as different graphs. Graph equivalence here means an isomorphism preserving the initial state, halt label, read symbol, write symbol, and direction. It is a substantially narrower relation than having the same halting behavior. In particular, a successful context-sensitive state fold need not be an ordinary graph quotient.

The eighteen incoming ZIP archives were inventoried. The comparison found twenty-seven accepted U208 presentations in twenty transition-graph classes. Nine originally designated finals belong to two classes, called A and B; the broader research archive contains eighteen further accepted helper variants, nine in each family. The helper variants are not simply dense renamings of the two representatives.

Nine supplied 196-state presentations lie in nine classes: Chang's original, an alternative head order, four auxiliary-decoder variants, two rotated layouts, and a source-factorization variant with a compensating scanner state. Together with PA209, this gives **thirty-seven supplied accepted presentations in thirty graph classes**. Here "accepted" describes the evidence retained in the supplied artifacts—literal table, executable checks, and the relevant source bridge—and does not mean that every variant has its own completed Lean development. Experimental modus-ponens variants that did not meet those conditions are excluded from this count.

Appendix B lists all thirty-seven presentations individually, with their state counts, exact graph-class assignments, and hash prefixes. The accompanying `catalogue.json` binds them to full hashes and table paths, and `catalogue.py` independently regenerates the combined census from the supplied tables.

Representative	States	Primitive sites	Internal DAG nodes	Maximum depth
PA209	209	3948	173	33
U208 A	208	3909	174	42
U208 B	208	3909	174	41
PA196	196	3812	159	38
Alternative PA196 layout	196	3812	159	36

Table 2: Source-tree measurements, with twenty-three leaf instruction types excluded from the internal-node count.

This distinction matters when reading the supplied scripts: one metric called `dag_nodes` counts internal nodes *and* distinct leaf types. A reported value of 181 can therefore mean 158 internal nodes, not 181 physical TM states. Table 2 uses internal nodes consistently.

U208 A and B have the same 5083 nonhalting source edges but differ at 720 instruction paths. PA196 has 4958 such edges and 4959 primitive instruction contracts when the halt case is included. A decrement has separate zero and positive contracts. These quantities describe the complete source control graph, not a bounded initial run.

1.2 What changed between the branches

The reductions do not combine by subtracting their headline state savings. The U208 branch and the PA196 branch use different source layouts and different sharing of physical states. A control can simultaneously read a source bit, implement a return path, and serve as a helper for a context-dependent decoder fold.

The published PA196 budget is 36 unchanged tape-engine controls, 13 leaf routers, and 147 reader controls implementing 159 distinct internal source nodes[1]. Its thirteen-state improvement over PA209 is concentrated in AX. The universal constructor changes from $4p(a, B)+2$ to $4p(B, a)+2$; the old seventh axiom head is removed and its required interfaces are derived using guarded equality vacuity added to the gate. The remaining heads are reordered, with several antecedent orders exchanged. The resulting AX body has 201 macro items, a 68-node reader grammar, and 62 physical reader controls after sharing. The existing sharing is already included in the published count and cannot be subtracted again.

The AX axiom-emission interval has 3779 primitive sites in PA209, 3742 in U208 A, and 3643 in PA196. Outside that interval, PA209 and PA196 have 169 primitive sites, whereas U208 has 167. Thus U208's thirty-nine-site reduction from PA209 consists of thirty-seven sites inside AX and two sites outside it. The two outside changes are independent of the axiom formula compression:

- A cyclic relocation moves the first twenty-five primitive instructions to the end and uses root wraparound in place of an explicit final continuation.
- Shift clears the old top directly instead of first copying it into the scratch register. Its falsity check tests the old head alone, instead of the sum of old top and old head.

These are meaningful candidates for combination with PA196's smaller AX. They still require a new physical decoder and complete runtime checks.

The often-quoted U208 deletion of native state 211 into native state 172 is tied to the entry contexts and helper native state 217 in that table. PA196 has different meanings and sharing at

the corresponding interfaces. There is no justified operation "delete the same-numbered state from PA196."

1.3 The published formal theorem boundary

The PA196 repository exports an arithmetic nonhalting sentence N and proves[1]

$$\mathsf{PA} \vdash N \longrightarrow \mathrm{Con}(\mathsf{PA}).$$

It also proves external nonhalting and the intended independence results, with the stated metatheoretic assumptions. The inspected PA196 and PA209 endpoints do *not* export a theorem

$$\mathsf{PA} \vdash \mathrm{Con}(\mathsf{PA}) \longrightarrow N.$$

External soundness in a hereditarily finite model is not, by itself, that internal converse. This is a statement about the inspected theorem exports, not a claim that the converse is false or unattainable. Section 3 explains the additional syntactic argument needed for the exact biconditional requested here.

The blank-tape startup of PA196 reaches its canonical source frame after 58,154 steps; PA209 requires 58,210. Both have head position -2, header distance 48, and the same physical register sequence

$$[0,0,0,0,0,0,2,0,0] +\!\!+ [1,\ldots,1] \quad \text{(twenty-six trailing ones)}.$$

The respective dense source-root states are 75 and 81. Startup checks compare the complete tape, not just the head, control, or number of ones.

2 New constructions that pass the complete checks

This section records concrete improvements obtained during this investigation. The two primary axiom-emitter constructions and a further scanner-oriented association have 196 working states. They improve the implementation of the alternative calculus found in the incoming research, but do not meet the requested strict inequality.

2.1 Diagonal vacuity in 196 states

The incoming diagonal-vacuity implementation has 199 states and an AX word of 191 macro items. The new implementation keeps that exact word and its logical meaning, changes its binary association, and performs six mutually compatible context folds. Its source program has 3700 primitive sites and 4815 zero/positive/ordinary instruction contracts.

Before folding, the construction retains 134 physical controls and allocates 68 AX reader controls. Six validated deletions give

$$134 + 68 - 6 = 196.$$

The reader associations matter because an instruction's binary address is also its program counter on the tape. This is not simply a compression of a string stored elsewhere.

Using the native labels in the construction files, the six deletions identify the following pairs:

Removed	Retained	Removed	Retained	Removed	Retained
2018	2024	2023	2019	2042	2028
2053	2032	2055	2056	2062	2036

The replacement rows and all redirects are explicit in the fold JSON. A local checker tests the final simultaneous table on both admissible parent-bit contexts and checks the endpoint control, head position, and complete two-cell tape. A separate whole-program checker then verifies every instruction contract. Checking each fold only against the original table is insufficient: one fold can change a helper used by another.

The resulting table passes all 4815 symbolic contracts. The longest checked symbolic path has 488 steps; the total is 1,503,668 symbolic steps. These are finite certificate sizes, not upper bounds on concrete unary execution times for arbitrary registers. The exact blank startup reaches the standard frame after 58,154 physical steps. An independent reconstruction reproduces the contracts and startup and finds all 196 controls graph-reachable. No two controls are identified by ordinary action bisimulation.

The dense table has initial state 0 and halt label 196. Its SHA-256 is

`59bbe3dbe040de4903b7f9b4fb64f70642757c54849e921a44af29145d625b0c`.

The written calibration of the diagonal calculus is inherited from the incoming work and explained in Section 3; the reduction from its 199-state realization to this concrete 196-state table is new to this investigation.

2.2 Separating the diagonal head: 190 macro items

There is a second concrete 196-state implementation. Emit $\Gamma(A) = A \to U(A, A)$ as an additional unconditional head, and leave the old gate $G = \neg(V \to Q)$ under its original guard. The extra implication which bundles Γ into the gate is no longer constructed. The AX word has 190 items and the entire primitive program has 3673 sites.

This version has ten pushed items, including the saved top, rather than nine. Consequently the proof-command selectors are adjusted: with Γ pushed immediately after the six retained heads, each old positive Shift count used to select those heads, the propositional head, generalization, or the saved top increases by one. The new head is available after two Shifts. The top-level gate is still available immediately. The stack operations and refutation compiler require these adjusted constants; one cannot substitute the new table and retain all nine-item selector identities verbatim.

The physical construction again has 68 AX reader nodes and six compatible folds above the same 134-control base. It passes all 4780 complete instruction contracts and the exact 58,154-step blank startup. The maximum symbolic path length is 488 and the total is 1,492,785 symbolic steps. The dense table SHA-256 is

`61ffdfea87d204cfb879ffb78fb1c9c47f425fdb53771995a40dbe0f0c9c3e28`.

The shortened macro word is an implementation improvement with a complete physical table. It is not a 195-state result.

2.3 Scanner-oriented 196-state layouts

A separate source-CFG-preserving rotation search produced a 196-state layout with fifteen addresses obstructing a proposed shorter origin scanner, down from nineteen in the original PA196 layout. All 4959 instruction contracts pass, as does the complete startup after 58,294 steps. The maximum symbolic path length is 497 and the total is 1,602,117 symbolic steps.

The proposed scanner edit would replace native transition $23, 0$ by 0L75 and delete native state 34. It would reduce the marker discrimination from seven zeros to six. The address obstruction

considered by this search is a substring `1000000`, or seven leading zeros; six leading zeros alone are allowed by that address test. The baseline-calculus layout still has fifteen obstructing sites, so this test does not justify applying the scanner deletion.

A second portfolio search applies the same objective to the separate-head calculus. It obtains a new 196-state table with 69 AX readers and seven mutually compatible folds:

$$134 + 69 - 7 = 196.$$

An exterior source-tree rotation preserves the complete source instruction CFG and removes the sole obstruction outside AX. The remaining internal obstructed source ordinals are 197, 584, 786, 867, 1759, and 2068. The unmodified scanner table passes all 4780 complete contracts, with maximum 515 and total 1,548,633 symbolic steps, and its exact blank startup reaches the frame after 58,294 steps. All 196 controls are graph-reachable. Its dense-table SHA-256 is

`bccb07e7da72d96806ab325cb321a88f28cdf0c14bfa87f9efd5e1a7d418c10a`.

The literal 195-state scanner edit was also constructed and tested. It fails 96 unrestricted branch obligations, including failures beyond the six address obstructions. It also fails the stronger invariant-conditioned simulation at source site 0. From the exact clean seed at source site 25, its first root return is after 118 steps, but its tape differs in 43 cells from the required site-27 successor frame. Thus the simple deletion does not implement the claimed clean-frame transition. These failures reject that simulation certificate; they do not by themselves decide the edited machine's actual blank-tape consistency property. No accepted smaller machine follows from this route.

2.4 What the executable certificates establish

The instruction checker starts at a complete symbolic frame, with arbitrary natural register values, an arbitrary permitted gap, and an arbitrary register-list tail. It proves the unchanged list/register routines by their affine-run induction templates, executes the finite decoder and instruction paths, and checks the entire successor frame. Decrements have separate zero and positive cases; the halt leaf is checked explicitly. The seed check is a separate exact simulation of the complete finite blank prefix.

Thus these checks go beyond sampling small registers or running the blank machine for many steps. They remain executable proof procedures whose soundness is described in the inherited verification material, rather than newly generated Lean kernel proofs. The package preserves this distinction in its receipts and reproduction instructions.

3 The precise PA equivalence

Let H_M be the arithmetic statement that the literal blank-tape machine M has a finite halting trace, and let $N_M = \neg H_M$. The requested property is

$$\mathsf{PA} \vdash N_M \leftrightarrow \mathrm{Con}(\mathsf{PA}). \tag{1}$$

Equivalently, PA must prove that a halting trace exists exactly when a PA refutation exists. Both transformations must be internal to PA, including their totality and correctness. External agreement on the standard natural numbers alone is a weaker conclusion.

The published PA196 formal development supplies the refutation-to-halting direction. The incoming research also contains a written syntactic construction for the other direction. We

summarize and sharpen that construction here, in particular the zero-stack issue and the positive-length reflection argument. This section is a mathematical proof construction; no new completed Lean endpoint is claimed for it.

3.1 Coding and arithmetic interpretation

Write

$$p(a,b) = \frac{(a+b)(a+b+1)}{2} + b.$$

The PA196 constructors are

$$I(A,B) = 4p(A,B), \quad M(a,b) = 4p(a,b)+1, \quad U(a,B) = 4p(B,a)+2, \quad E(a,b) = 4p(a,b)+3.$$

Code 0 is truth, and code $1 = M(0,0)$ is falsity. Negation means $I(A,1)$; conjunction and existential quantification are their usual classical abbreviations. For every nonzero code, each recursive formula subcode is smaller. Every variable label occurring in a decoded formula of code c is strictly smaller than c. These numerical facts make decoding primitive recursive and make x_c fresh in formula c.

In a conservative definitional extension of PA by the provably total function 2^n, define

$$\mathrm{Bit}(s,z) \quad \Longleftrightarrow \quad \exists u, v\,[s = (2u+1)2^z + v \ \wedge \ v < 2^z].$$

PA proves $\mathrm{Bit}(s,z) \to z < s$, hence $\neg\,\mathrm{Bit}(x,x)$. Translate membership with its *first* argument as the container:

$$T(0) = (0=0), \qquad T(M(a,b)) = \mathrm{Bit}(x_a, x_b),$$
$$T(E(a,b)) = (x_a = x_b), \qquad T(U(a,B)) = \forall x_a\, T(B).$$

Implication decoding is accompanied by a uniformly generated proof of $T(I(A,B)) \leftrightarrow (T(A) \to T(B))$. Its exceptional case $I(0,0) = 0$ uses the fixed proof that truth is equivalent to truth implying truth. Auxiliary bound variables in the bit formula are renamed effectively to avoid capture. The source freshness lemma concerns the source labels and remains valid after this renaming.

3.2 Proof templates for finite separation

For every arithmetic formula $\varphi(z, \vec{u})$, PA proves

$$\forall n\, \exists s < 2^n\, \forall z < n\, \big(\mathrm{Bit}(s,z) \leftrightarrow \varphi(z,\vec{u})\big). \tag{2}$$

The induction starts with $s = 0$. At stage n, use excluded middle on $\varphi(n,\vec{u})$ and choose $s + 2^n$ or s. Elementary binary arithmetic verifies the old bits and the new bit. Full PA induction is essential: φ may have arbitrary quantifier complexity.

There is a primitive-recursive procedure which substitutes the syntax of φ into this proof template and emits a PA proof. It does not evaluate φ. The uniform statement to be proved is about the output being a valid proof code, not about a uniform truth predicate. A fixed primitive-recursive elimination of the definitional extension returns an ordinary PA proof.

For set codes A, B, apply (2) with bound $A + B + 1$ and matrix

$$(\mathrm{Bit}(A,z) \vee z = B) \wedge \varphi(z).$$

The resulting code s satisfies

$$\forall z\,[\mathrm{Bit}(A,z) \to (\varphi(z) \to \mathrm{Bit}(s,z))],$$
$$\forall z\,[\mathrm{Bit}(s,z) \to \varphi(z)], \qquad \varphi(B) \to \mathrm{Bit}(s,B).$$

The unbounded universal conclusions follow because elements of A lie below A and s has no set bits beyond the chosen bound. This is the pointed separation required by the source gate.

For source labels a, b and a formula code c, the PA196 gate is

$$\Delta = E(a,b) \to U(c, E(a,b)), \qquad V = M(a,b) \to U(c, M(a,b)),$$
$$P = c \to M(c,b), \qquad S = U\big(b, (M(a,b) \to P) \wedge (M(c,b) \to c)\big),$$
$$Q = U(c, S \to \neg P), \qquad F = \neg(\Delta \to (V \to Q)).$$

When $c \neq a, b$, the two vacuity clauses have first-order proofs. Apply the separation template to the capture-avoiding substitution of x_b in $T(c)$. This proves an existential instance of $T(S) \wedge T(P)$ and therefore $\neg T(Q)$ and $T(F)$. If $a = b$, the first antecedent inside S becomes $\mathrm{Bit}(z,z)$ and is false; the same comprehension witness proves the other conditions. Thus this case does not require a stronger guard. If the guard fails, the command emits truth and uses its fixed proof.

The other heads are instances of first-order logic with equality, and the restricted propositional head is a classical tautology. Each has a primitive-recursive proof supplier. Modus ponens and generalization have their usual syntactic proof constructors. Consequently one obtains a primitive-recursive transformation R with

$$\mathsf{PA} \vdash \forall p\,\big(\mathrm{Prf}_{\mathrm{source}}(p,1) \to \mathrm{Prf}_{\mathsf{PA}}(R(p), \ulcorner 0 = 1 \urcorner)\big). \tag{3}$$

The proof is induction on the coded source derivation, using the template correctness lemmas. It asserts no reflection principle for PA.

3.3 The proof-store invariant and the zero alias

The command store has two visible formula codes t, h and a natural-number stack code w. Annotate both visible formulas and all live stack entries with source derivations, or directly with their translated PA proofs. AX pushes the available axiom proofs and a generalization of the old head. A successful modus-ponens match constructs a new proof; a failed match returns the fixed truth proof. Shift transports these proof annotations with the formula values. Its original falsity test is $t + h = 1$, so tracking t alone is insufficient.

An annotation of an ordinary abstract list must account for $p(0,0) = 0$. Pushing truth onto an empty numerical stack leaves the stack code zero; it does not create a distinguishable live entry. Use the canonical expansion of w instead. For $w > 0$, the second Cantor component satisfies $\mathrm{tail}(w) < w$; for $w = 0$, let both components be zero. Define the live depth as the least $k \leq w$ with $\mathrm{tail}^k(w) = 0$.

The annotation operation is then exact:

- Pushing a onto w prepends its proof unless $a = w = 0$, in which case the canonical annotation remains empty.
- Popping a nonzero stack removes the first proof annotation.
- Popping zero returns the fixed truth proof and the empty annotation.

All these operations and their invariants are primitive recursive, with their defining inductions available in PA. Thus a source halt supplies a proof of code one and (3) supplies a PA refutation.

3.4 Reflecting physical halting back to the source

The runtime obligation is stronger than "a source step is simulated unless the physical machine happens to halt." Each nonhalting source step must have an *exact*, positive-length physical execution segment to the next complete frame, without a premature halt. The segment records the whole tape and preserves the arbitrary register tail and gap. The register excursions have PA-provable terminating bounds.

After the exact finite startup, suppose a physical halting trace has length T. Execute $T+1$ source instructions from the clean frame. If none halts, concatenate their exact positive-length physical segments. The resulting physical execution is longer than T and never halts in the middle. Determinism contradicts the alleged first halting trace. Hence one of those source instructions halts, and its proof-store annotation gives a PA refutation.

This is a bounded coded iteration argument inside PA: source stepping, segment bounds, and finite concatenation have primitive-recursive definitions, and PA proves the needed inductions. Together with the published refutation-to-halting compiler, it establishes the mathematical construction for (1). The executable runtime certificates in this archive are finite representations of the required symbolic cases. Turning their soundness and the strengthened reflection statement into additional Lean declarations remains separate formalization work.

3.5 Why diagonal vacuity preserves PA strength

The earlier incoming research replaces Δ by

$$\Gamma(B) = B \to U(B, B).$$

The code-freshness lemma makes this sound under T. The guarded gate becomes $F_\Gamma(a, b, c) = \neg(\Gamma(c) \to (V \to Q))$. It yields both $\Gamma(c)$ and the old gate $G = \neg(V \to Q)$. For any B, choose $c = B$ and $a = b = B + 1$; the guard succeeds and yields $\Gamma(B)$.

For completeness, the affected equality interfaces must be recovered, rather than simply asserted. The direct membership rewrites and equality witness head are retained. To recover a reverse rewrite, set

$$C = E(a, b) \to (\chi(b) \to \chi(a)), \quad \varphi = \neg C, \quad c = \ulcorner \varphi \urcorner,$$

where $\chi(t)$ is $M(d, t)$, $M(t, d)$, or $M(t, t)$. Numeric freshness makes c different from all relevant labels. Pointed separation at (a, a, c) supplies a witness with point condition $P = \varphi \to M(c, a)$ and $S \to U(a, M(c, a) \to \varphi)$.

Under φ one has $E(a, b)$, $\chi(b)$, and $\neg\chi(a)$. First weaken the last quantified consequent to $\neg\chi(a)$. The equality atom inside φ has thereby been removed before applying the negative-property specialization lemma. That lemma uses only membership vacuity, the equality witness, and the direct rewrites, and gives

$$S \to (M(c, b) \to \neg\chi(b)).$$

Under φ, the point condition and direct rewriting give $M(c, b)$, contradicting $\chi(b)$. Thus $\varphi \to (S \to \neg P)$. Generalize at c and use $\Gamma(\varphi)$ to obtain $\varphi \to Q$. Since the old gate gives $\neg Q$, classical reasoning gives C. Equal-label cases are either propositional or reduce to the diagonal membership case. This avoids a circular use of equality vacuity in the reverse-rewrite proof.

For neighborhood equality, choose $z_0 = \ulcorner E(a, b) \urcorner$. The diagonal clause for $E(a, b)$ and the recovered two-direction rewrites give

$$E(a, b) \to (U(z_0, M(a, z_0) \leftrightarrow M(b, z_0)) \ \wedge \ U(z_0, M(z_0, a) \leftrightarrow M(z_0, b))) .$$

For any $z \neq a, b$, transfer the binder from z_0 to z using the equality witness and the two-direction rewrites. The matrices and the transferred antecedent contain only membership atoms, so the

required vacuity uses no equality-vacuity premise. This recovers the original neighborhood interface at every permitted label, not merely at numerically large labels. The witness interface is unchanged.

The affected proof templates are finite and their choices of fresh labels are primitive recursive. Substituting these templates in the original refutation compiler preserves its internal PA correctness. Soundness uses the proof supplier above plus the freshness proof for Γ. Separating Γ as an unconditional extra AX head while leaving G guarded gives the same calculus: the bundled gate and its separated components are interderivable by propositional reasoning and the same guarded extraction.

3.6 The simpler Shift used by U208

For the hybrid investigation, define

$$S_h(t, w, d, h) = \begin{cases} \text{halt}, & h = 1, \\ (h, \text{tail}(w), d, \text{head}(w)), & h \neq 1. \end{cases}$$

This differs from the original check $t + h = 1$. The proof-store soundness argument remains valid because a halt still exposes a proved code one.

For the nine-item AX of PA196, eight successive S_h operations after AX expose the same pair as eight original Shifts: top ρ_8, head $U(d, h_{\text{old}})$, and stack $p(t_{\text{old}}, w_{\text{old}})$. Nine expose top $U(d, h_{\text{old}})$ and head t_{old}. Ten give the old pop result unless $t_{\text{old}} = 1$, in which case the revised command halts. The emitted fixed heads and generalization cannot themselves be code one, by their constructor tags and nontrivial implication shapes.

The stack and modus-ponens compiler identities therefore retain their "halt or the specified result" form. At the final result $t = 1, h = 0$, append AX with zero parameters followed by ten Shifts: nine expose the saved falsity in h and the tenth halts. This supplies the small logical adaptation required by the hybrid. It does not solve the physical blank-start obstruction reported in the experimental section.

3.7 What ConciseIPRA contributes

The ConciseIPRA project concerns presentations bi-interpretable with full first-order intuitionistic primitive recursive arithmetic. Its native language uses an iterator and equality, with derived logical operations and compact closure and choice rules. Its reported Python token, AST-node, and byte counts measure a complete proof checker in that programming model. They are not TM state counts.

The transferable strategy is to replace a primitive rule by a smaller set of derivably equivalent interfaces, to infer information from syntax where possible, and to measure the complete implementation. The diagonal replacement above follows that broad strategy, but its specific code-freshness trick was already present in the incoming PA-machine research and should not be misattributed to ConciseIPRA.

A refutation search for iPRA alone would target Con(iPRA). It does not establish (1). Any route through a concise intuitionistic checker must preserve the full induction strength needed for a PA refutation translation, for example by proving the appropriate equivalence for a full-induction extension and then the requisite intuitionistic/classical translation. No standalone sub-196 implementation of such an extension is claimed here.

4 Why the attempted additional state savings were rejected

Negative searches in this section have explicitly bounded templates. They do not establish a lower bound of 196 on PA-consistency machines. A valid smaller machine may require a different encoding, program, invariant, or simultaneous edit outside these templates.

A failed contract on arbitrary source values, or on values admitted by an abstract invariant, refutes that proposed simulation proof. The admitted values can overapproximate actual reachability, so such a failure does not alone disprove the edited machine's property from blank tape. The explicit outward-loop startup traces below are a different kind of evidence: they describe the actual blank computation. The literal short-scanner edit also has a concrete failure from the intended clean source seed, while its own blank-start behavior remains unproved.

4.1 An optimistic 195-state count was not a construction

A diagonal AX association had seventy raw reader nodes and nine individually available pair identifications. Subtracting nine from seventy suggested sixty-one reader states and hence $134 + 61 = 195$ total states. That count failed two distinct checks. Some macro occurrences entered through a side on which their existing physical implementation was not valid. After correcting those entries, several selected folds interfered with helpers used by other folds.

The replacement compatibility checker applies the proposed redirects simultaneously and replays the local obligations against that final table. The surviving jointly compatible set was smaller, and the optimistic 195-state table was rejected. The accepted 196-state tables in Section 2 pass the stronger check and then the complete runtime check.

The search objective was also corrected to reuse native controls 99 and 190 for the existing two- and four-increment reader patterns. Treating the corresponding literal binary tuples as newly allocated grammar nodes overcharges them. Mixed-integer grammar searches additionally considered repeated substrings and free existing routines. Their raw grammar costs are exploratory objectives until entry sides, scanner bounds, helper compatibility, and startup have all been checked.

The corrected all-helper search completed 160 cycles of 1600 attempted rotations, for 256,000 proposals. Its best effective AX cost remained 62, corresponding to 196 total states above the 134-control base. This is a heuristic search, not an exhaustive enumeration of binary trees. The scanner portfolio completed a further 30,000 attempted proposals and reached the six-obstruction layout in Section 2. Exact saved inputs and logs accompany the code. Finite follow-up screens on 98 saved cost-62 grammars and the scanner grammar found no additional action-bisimulation merge or compatible second folding round.

4.2 Restricted allocation has explicit Hall obstructions

Several incoming source layouts have only 157 or 158 internal nodes. A fixed-reader allocator attempts to retain recognized subtrees, protect the rows needed by the register engine and contextual helpers, and injectively assign the remaining new source nodes to available physical labels. Removing an exploratory ordering constraint from that allocator does not make these particular domains feasible.

For each displayed Hall witness, the union of the allowed-label domains of the listed source nodes is smaller than the set of nodes. Therefore no injection satisfying those domains exists. The JSON receipts include the actual source paths and labels, and the verifier recomputes their domain union. This is stronger than a solver timeout but much narrower than an impossibility theorem about the source program or all 195-state machines.

Layout	New nodes	Maximum matching	Hall nodes	Available labels
`source157`	81	75	81	75
`essential158`	26	23	24	21
`layout158`	46	38	44	36

Table 3: Finite obstructions to the specified injective allocation model.

4.3 The U208/PA196 hybrid and its startup obstruction

The source hybrid grafts PA196's AX into U208 A's outside program. It has 3810 primitive sites and 159 internal DAG nodes. A prefix-only allocator can bind this source to 196 controls, but its first allocation failed a complete primitive contract at site 23. Passing a decoder-prefix test is therefore not sufficient.

A second construction retains the actual U208 outside implementation and transplants the PA196 AX implementation through exact interface bindings. Initially the complete required table has 197 controls. An outside helper uses only its read-one row in all outside contracts, and that row matches a copied AX helper. Reusing its other row yields a 196-control table. The resulting table passes all 4957 arbitrary-value instruction contracts; all 392 transition rows occur in those contracts.

It nevertheless fails as a blank-start consistency machine. From native initial state 0, after 132 steps it reaches an outward rightward loop in native state 35 on untouched blank tape. Starting in native state 74 reaches the same obstruction after 54 steps. A finite screening of every possible existing initial control finds an explicit outward blank self-loop in each case. This is an actual startup failure, not merely an unfinished long simulation. The table is a correct interpreter of supplied source frames but is not the requested blank-tape machine.

The first divergence from the original U208 blank execution is at step 124, native state 11 reading one at head position 4. The old AX prefix consumes address bits `011` to enter the existing paired-increment control 99. The transplanted AX has a different association at that prefix. This identifies the lost bootstrap behavior precisely. Enforcing that prefix by reassociation restores an exact source frame after 47,068 steps at the original header distance 56, but the straightforward repair costs an extra control and introduces two seven-zero addresses. A complete runtime test of site 842, decrementing register 1 in its zero case, fails. The repaired 197-state table therefore lacks the claimed runtime certificate, despite its successful finite startup; that failed arbitrary-frame obligation does not decide its actual blank behavior.

The diagonal and separated-diagonal AX versions likewise give 196-state candidate interpreters when grafted into U208's outer layout; their sources have 3698 and 3671 sites. Construction and prefix binding pass, but their full runtime contracts were not rerun. Their initial-control screens already fail before a clean frame. Thus neither shorter logical word by itself repairs the bootstrap.

4.4 Bounded physical state-fold searches

The baseline source has 4959 complete instruction contracts. A stronger local enclosure projects these contracts to nine tape cells, four on either side of the head, and combines them with the independent local blank-window enclosure. It admits 12,168 control/window pairs. The synthesis template identifies one ordered pair of states and permits two arbitrary changed rows in retained states, with positive simulated blocks of at most 32 steps and exact endpoint control, head, and all nine tape cells.

All 38,220 ordered baseline folds were screened. The run made 174,570,928 first-row action trials

and 1,866,384,912 second-row action trials. Of 803 representative-filter passes, none survived all obligations. Counts concern trials of the template, not distinct machine isomorphism classes.

A separate limited three-row search extended the 633 representative passes from the earlier five-cell, sixteen-step two-row screen. The third changed row had to lie on every remaining failed trace. It tested 1400 eligible third-row keys and 1,097,600 actions, without a survivor. It does not cover three-row candidates whose first two repairs fail that representative filter.

Fresh two-row screens were also run on two previously accepted rotated PA196 layouts, with 2223 and 2217 five-cell contexts. Their complete contracts and startup were rechecked before searching. These bounded screens produced no additional fold. The exact receipts specify the additional new layouts and completed scopes; interrupted runs are not counted as exhaustive negatives.

4.5 Head-offset state identifications

Another template permits a removed state q to be represented by a retained state p with a two-cell head offset. If every incoming transition has direction $d \in \{-1, 1\}$, redirect it to p and reverse its direction. The child head is then $-2d$ relative to the parent head when the parent is in q, with identical tape. Outside q, the heads and controls agree. Modified helper states must have positive exact simulation blocks restoring the relation.

All 33,345 uniform-incoming ordered pairs failed without an additional row repair. With one arbitrary retained-row repair, 44,853 eligible keys and 35,164,752 actions were tested, again without a survivor. The remaining 4875 mixed-direction pairs were screened under a relation remembering the incoming direction; 246,176 single-helper actions produced no survivor. These screens use nine cells and at most 64 simulated steps.

The stronger two-helper uniform-offset screen also completed on all 33,345 pairs: 179,423,888 first-row and 14,854,027,776 second-row action trials produced no survivor. These are the counts of this bounded synthesis procedure, not a count of all possible 195-state machines.

A separate six-state positive control recovers the expected deletion of state 3 into state 4 at offset -2, on all 512 nine-cell windows at every state. That test detects mistakes in the nontrivial offset handling. It does not establish completeness of the PA196 search beyond the stated template.

4.6 Finite register invariants

A trit abstraction records each logical register as zero, one, or at least two. Increment and decrement use their exact sound abstract transfers; a decrement from "at least two" may produce either one or at least two. From the canonical source seed, the new diagonal program reaches 4,319,427 abstract site/register pairs. A separate closure audit checks 4,848,294 abstract edges.

The invariant proves a few branches impossible and validates several alternative closing blocks. The explicit halt site remains abstractly reachable. In particular, the unreachable site 8 is a continuation, not the halt instruction. This finite analysis is not a proof of the machine's nonhalting and does not contradict the required incompleteness phenomenon.

The site number matters: the zero condition at source site 3661 does not justify deleting the different decrement at site 3648. A proposed deletion of native reader 174 based on that mistaken correspondence was withdrawn. The corresponding 195-state edit is not an accepted construction.

Projecting the finite invariant to the nine register coordinates yields sound marginal ranges at each site. The full affine checker verifies 5149 restricted cases. Marginals alone leave the strongest previous five-cell context set unchanged at 2210 windows. A separate induction gives a useful tape invariant: the startup has twenty-six spare blocks of value one, and primitives preserve

those values while increments append a further one-valued spare. Instantiating the first spare as one and checking the periodic interiors of the remaining list reduces the admitted set to 2173 windows. The complete 58,154-step startup is included separately. The accompanying source notes explain why testing twenty-five periodic blocks covers every longer list for these observation radii; this is an unbounded periodicity argument rather than an assumed finite spare limit.

At radius four, the corresponding nine-cell enclosure shrinks from 11,962 to 10,749 control/window pairs. Its completed two-row, 32-step screen again covers all 38,220 ordered folds: 174,648,544 first-row and 1,904,561,008 second-row trials leave no survivor after 797 representative-filter passes. The corresponding 64-step uniform-offset screen covers all 33,345 ordered pairs and 35,668,864 helper actions, again without a survivor. Completed five-cell screens and four alternative closing-block screens likewise produce no accepted deletion; their exact inputs and receipts are retained.

The separate-head source used by the final scanner experiment has 4,270,098 reachable abstract pairs and 4,792,890 audited abstract edges. Its halt remains abstractly reachable. Even after intersecting the short-scanner obligations with those trit marginals and the one-valued-spare invariant, the site-0 zero-decrement obligation fails. This sharper invariant therefore does not repair the proposed 195-state simulation proof.

5 Archive contents and reproduction

The ZIP contains the accepted constructions and retained finite checks. Repository inputs are pinned to exact commits; inherited licenses and attribution are preserved beside them.

Path	Contents
`report/`	Complete LaTeX sources, compiled PDF, and literal diagonal196 table.
`axioms/`	Three new 196-state layouts; source, grammar, and fold certificates; complete symbolic runtime and startup checks; logical proofs; completed search inputs and logs.
`comparison/`	Pinned supplied tables, census, baseline scanner layout, hybrid experiment, and archive-member byte bindings.
`engine/`	Physical reduction code, contexts, invariants, positive controls, and bounded-search receipts.
`allocation/`	Fixed-reader allocator, source inputs, and replayable Hall witnesses.
`SHA256SUMS`	Hash manifest for all packaged files except the manifest itself.

5.1 Accepted constructions

Run Python with assertions enabled. From the archive root:

```
python3 axioms/verify.py
python3 catalogue.py
python3 comparison/comparison/compare_designs.py --output comparison_replayed.json
python3 comparison/scanner196/verify.py comparison/scanner196 --startup 150000
cd axioms/search
python3 export_scanner.py scanner_6 scanner_best196
cd ../..
```

The axiom verifier regenerates the literal tables and dense renamings, checks macro control flow and emitted constructor trees, checks every instruction contract, and replays the complete blank

startups. It requires only the Python standard library. Comparison and scanner checks are separate.

5.2 Experimental searches and allocation certificates

```
python3 engine/reproduce.py
python3 engine/reproduce_trit.py
python3 engine/check_scanner195.py
python3 allocation/verify_hall.py
```

The default engine command checks digests and positive controls. Its `--full` option reruns the completed searches using a C++17 compiler. The trit gate reconstructs the source graph using three resolvers, reruns the fixed point and closure audit, and checks invariant/context bindings; its `--full` mode also requires NumPy. The scanner195 check reproduces the expected failed contracts. Raw receipts specify the exact templates, inputs, bounds, and trial counts.

The Hall verifier regenerates the domains and witnesses using only Python's standard library. These three obstructions are decisive before a solver is invoked. Exploring other allocations requires `z3-solver`. Inputs and hashes are pinned, and the proof is restricted to that allocation model.

5.3 Rebuilding the PDF

```
cd report
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The document uses ordinary TeX Live packages, including Latin Modern, AMS mathematics, `booktabs`, `microtype`, and `hyperref`. The PDF was compiled and visually inspected after rendering.

5.4 Evidence levels and limitations

The following distinctions are maintained throughout the package:

1. A raw grammar count, solver objective, or successful decoder-prefix test is an exploratory result.
2. A complete symbolic instruction certificate plus exact startup establishes the stated physical implementation, relative to the verified symbolic block methodology.
3. The PA calibration additionally requires the logical proof translations and their arithmetization; Section 3 spells out the written construction.
4. A new theorem accepted by the Lean kernel is a separate deliverable. No Lean installation was available for a new build in this environment, and no such new endpoint is represented as completed.

The strict target remains unachieved. The archive preserves the constructions and failed paths without promoting rejected candidates to accepted machines.

A Literal diagonal196 transition table

The initial tape is all zero, the head begins at coordinate zero, and the initial working state is 0. There are exactly 196 working states, numbered 0 through 195. The halt label H is represented by 196 in the machine file and has no outgoing transitions. An entry `wDq` means write bit `w`, move one cell in direction `D`, and enter state `q`. The two halves of each row list independent working states. This appendix is generated from the exact dense table with the SHA-256 printed in Section 2.

State	Read 0	Read 1	State	Read 0	Read 1
0	1R1	1R61	98	0R91	1R107
1	1R82	1R1	99	0L131	0L85
2	0L2	1L3	100	0R108	1R94
3	0L4	1L3	101	0L109	0L2
4	0L6	1L3	102	0R38	1R68
5	1R7	0L5	103	0R59	1R110
6	0L9	1L3	104	0R59	1R111
7	0R7	0R10	105	0R66	0L16
8	0R14	1R13	106	0R14	1R110
9	0L15	1L3	107	0R21	1R112
10	0L16	0R10	108	0R113	1L20
11	0L26	1R18	109	0R30	1R65
12	0R19	1R20	110	0R38	1L108
13	0L79	1L31	111	0R121	0L16
14	0L11	0R22	112	0R114	0L16
15	0L23	1L3	113	0R24	0R47
16	1L2	0L2	114	0R48	0R32
17	0R24	0R32	115	0R70	1R86
18	0R84	0L95	116	0R19	1R118
19	0R28	1R29	117	0R126	1R28
20	0R30	1R31	118	0R122	1R107
21	0R24	0R57	119	0R94	1R86
22	0R33	1R22	120	0R83	1R98
23	0L34	1L3	121	0R87	1R87
24	0L35	0R1	122	0R17	1L109
25	0R36	1R25	123	0R70	1R101
26	0R122	1R41	124	0R96	1R108
27	0R38	0L39	125	1L39	1L99
28	0R40	1R41	126	0R80	1R78
29	0R42	1R43	127	0R120	1R133
30	0R44	1R45	128	0R124	1R123
31	0R21	1R46	129	0R123	1R128
32	0R47	1R32	130	0R44	1R90
33	0R32	1R33	131	0L5	0L131
34	0L69	1L3	132	0R127	1R59
35	1R35	1L2	133	0R11	1R37
36	0R49	1R36	134	0R135	1R186
37	0R28	1R20	135	0R136	1R140
38	0L11	0R36	136	0R12	1R137
39	0L5	1L51	137	0R86	1R138
40	0R24	0R25	138	0R139	1R58
41	0R13	0L16	139	0R78	1R58
42	0R53	1R54	140	0R141	1R176
43	0R55	1R56	141	0R142	1R143
44	0R24	0R52	142	0R64	1R12
45	0R58	1R59	143	0R144	1R175

State	Read 0	Read 1	State	Read 0	Read 1
46	0R60	0L16	144	0R86	1R145
47	0R25	1R47	145	0R146	1R170
48	0L35	0R61	146	0R147	1R162
49	0R16	1R49	147	0R78	1R148
50	0R63	1R64	148	0R149	1R161
51	0L39	1L51	149	0R126	1R150
52	0R22	1R52	150	0R151	1R156
53	0L11	0R49	151	0R156	1R153
54	0R58	0L16	152	0L146	1R78
55	0R65	1R66	153	0R154	1R155
56	0R59	1R67	154	0R116	1R152
57	0R52	1R57	155	0R152	1R136
58	0R48	0R25	156	0L143	1R152
59	0L28	1L68	157	0R158	1R159
60	0L102	1L30	158	0R159	1R137
61	0R0	1R61	159	0R19	1R160
62	1R125	1L51	160	0R58	1L192
63	0R70	1RH	161	0R162	1R164
64	0L126	1R31	162	0R12	1R163
65	0R40	1R60	163	0R116	1R138
66	0L79	1R71	164	0L179	1R166
67	0R53	1L72	165	0R137	1R136
68	0R73	1R74	166	0R167	1R168
69	0R69	1R75	167	0R163	1R78
70	0R76	1R77	168	0L146	1L175
71	0R79	1L110	169	0R86	1R139
72	0L98	1R54	170	0R171	1R172
73	0R48	0R33	171	0R172	1R154
74	0R81	0L41	172	0R173	1R58
75	0R50	1R132	173	0R160	1R155
76	0R130	1R21	174	0R116	1R169
77	0R101	1R90	175	0R157	1R174
78	0R92	1R93	176	0R177	1R184
79	0R73	1R72	177	0R178	1R166
80	0R91	1R119	178	0R58	1R179
81	0R95	1R96	179	0R154	1R180
82	0R62	0R1	180	0R165	1R181
83	0R27	1R97	181	0R152	1R182
84	0R89	1R88	182	0R116	1R183
85	1L99	1L85	183	0R164	1R168
86	0R19	1R100	184	0R185	1R185
87	0R58	1R58	185	0L153	1R174
88	0R117	1R115	186	0R187	1R195
89	0L26	1R134	187	0R188	1R193
90	0R95	0L131	188	0R116	1R189
91	0R102	1R103	189	0R190	1R191
92	0R67	1R56	190	0R168	1R78
93	0R55	1R104	191	0R158	1R192
94	0R21	1R105	192	0R185	1R190
95	0R21	0L10	193	0R167	1R194
96	0R59	1R41	194	0R168	1R122
97	0R106	1R8	195	0R129	0L131

B Complete supplied-design catalogue

The identifiers below match the comparison artifacts. Classes preserve the initial state, halt, read/write symbols, and move directions. The abbreviated hash column is for navigation only; `catalogue.json` records complete SHA-256 values, original archive members, and exact local table paths. This catalogue excludes the new constructions of Section 2.

Presentation	States	Class	SHA-256 prefix
u208_basic	208	C00	9f4ea95849c5e1
u208_conditional_alternative	208	C01	9865dcd7bd44de
u208_conditional	208	C00	a2fee9e41311c0
u208_entry	208	C00	a2fee9e41311c0
u208_entry_alternative	208	C01	9865dcd7bd44de
u208_history_alternative	208	C01	7ba8ba5d5a5394
u208_history	208	C00	926d15c43daebd
u208_research	208	C01	5d487fb3ea29a8
u208_research_alternative	208	C00	b490b7dbe71b87
pa196	196	C02	e8bbccd4cfa325
pa209	209	C03	0610d978675320
u208_helper_A_0	208	C04	adb3726c233ec5
u208_helper_A_1	208	C05	b3c14c9fbb3489
u208_helper_A_2	208	C06	6d5b91e248e4b8
u208_helper_A_3	208	C07	f02bf11aa3207c
u208_helper_A_4	208	C08	d28e8e14666157
u208_helper_A_5	208	C09	4ee65d77aed40b
u208_helper_A_6	208	C10	44800fd752ce6d
u208_helper_A_7	208	C11	819a83ba50a8be
u208_helper_A_8	208	C12	b859e6c6c97523
u208_helper_B_0	208	C13	c271b3f08d9538
u208_helper_B_1	208	C14	2eab8a161d1e29
u208_helper_B_2	208	C15	04ff71fbdf73cc
u208_helper_B_3	208	C16	641407ba2c0ca0
u208_helper_B_4	208	C17	a736ccf9958eff
u208_helper_B_5	208	C18	a2fb9f02d02fcd
u208_helper_B_6	208	C19	d8834214354e84
u208_helper_B_7	208	C20	b7769f82c0b699
u208_helper_B_8	208	C21	dd34cb2ab6504b
pa196_incoming_alternative	196	C22	aa05f079326ae2
pa196_factorization_scanner	196	C23	1ab0fd1fe7b727
pa196_close_cd6	196	C24	2db09bb057682b
pa196_close_cd3	196	C25	d5c1c2c6908297
pa196_close_cd5	196	C26	4433af83b56864
pa196_close_cd4	196	C27	2be1986cbbac0c
pa196_rotated	196	C28	25d686ab95f58b
pa196_rotated_macro62	196	C29	591f4240775256

References

[1] James Chang, *BusyBeaver: PA196*, repository and paper, revision `407cdc356f6b586635e71ac92be394255f4bfea4`. https://github.com/jamesyc/BusyBeaver/tree/407cdc356f6b586635e71ac92be394255f4bfea4. The principal inspected files are `paper/pa196.tex`, `docs/result.md`, `PA196/Program.lean`, `PA196/Logic/Basic.lean`, `PA196/Logic/Soundness.lean`, and `PA196/Arithmetic/RefutationHalting.lean`.

[2] James Chang, *BusyBeaver: PA209*, revision `69d98241a721ead20b4f0b17df9062950b40fbfe`. https://github.com/jamesyc/BusyBeaver/tree/69d98241a721ead20b4f0b17df9062950b40fbfe.

[3] Vladimir Reshetnikov, *ProveIt, docs/incoming*, revision `dc523e7b06ffb414093c90aa035500ca283e5f75`. https://github.com/VladimirReshetnikov/ProveIt/tree/dc523e7b06ffb414093c90aa035500ca283e5f75/docs/incoming. Eighteen ZIP archives were inventoried. Exact archive names, member paths, and SHA-256 bindings are recorded in the accompanying comparison receipts.

[4] *PA consistency research*, incoming archive `pa_consistency_research_2026-11-13.zip`, especially `sections/diagonal.tex`, `sections/internal.tex`, and the `diagonal199/` artifacts, at the ProveIt revision of [3]. The diagonal-vacuity calculus and the earlier written reverse-implication construction originate in this supplied work.

[5] *PA196 research* and *PA TM comparison and reductions*, incoming archives `pa196_research_2026-11-13.zip` and `pa_tm_comparison_and_reductions.zip`, at the ProveIt revision of [3]. These supply the original comparison, local-fold templates, allocator, and symbolic runtime machinery adapted for this report.

[6] Vladimir Reshetnikov, *Concise presentations of intuitionistic primitive recursive arithmetic*, ProveIt, `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`, revision of [3]. https://github.com/VladimirReshetnikov/ProveIt/tree/dc523e7b06ffb414093c90aa035500ca283e5f75/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA. The project's README distinguishes its written results, executable checker measurements, Lean formalizations, and open questions.